



VectorSec



Euler

Penetration Testing report

May 23rd, 2026

Contents

Executive summary	3
Strategic recommendations	6
Penetration testing.....	6
Secure SDLC	6
Defense in depth	6
Vulnerability details.....	8
Medium severity findings.....	9
1.1 TRM screening race condition	9

Executive summary

Overall risk posture

The application's risk posture is assessed as **strong**. No vulnerability was identified that allows an unauthenticated remote attacker to directly compromise user funds, take over accounts, or read or modify arbitrary user data on the production deployment as it stands today. One medium issue was identified as a client-side race condition in the wallet connection flow that, in the narrow window before the TRM screening verdict returns, allows a sanctioned wallet to interact with the wallet-gated UI.

Risk posture ratings	
Critical	Application is actively exposed to direct compromise of user funds, accounts, or regulatory standing, and remediation should be treated as an incident-response activity rather than a planned engineering effort.
Weak	One or more findings allow an unauthenticated or low-privilege attacker to bypass meaningful controls, expose sensitive data, or undermine compliance enforcement. The surrounding infrastructure does not adequately compensate for these risks.
Adequate	Application is broadly defensible in its current state, with no finding allowing direct compromise of funds or accounts, though specific weaknesses remain that should be addressed to prevent erosion of the posture over time.
Strong	No finding materially threatens user funds, user data, or compliance standing. The controls at the infrastructure edge and within the application reinforce each other under adversarial conditions.

Project objectives and scope

This engagement was a penetration test of Euler's lite web application, commit hash: 199f8c481f95f6c4348713d2f9e792ecf2c40036. All tested API endpoints:

- /api/rpc/[chainId]
- /api/screen-address
- /api/tenderly/simulate
- /api/tenderly/status
- /api/pyth/updates
- /api/labels/[file]
- /api/oracle-adapter
- /api/euler-chains
- /api/token-list
- /api/intrinsic-apy/[provider]
- /api/tos

VectorSec Penetration Testing Report

The objectives were to identify vulnerabilities that could be exploited to compromise user funds, bypass sanctions and compliance controls, abuse paid upstream services at the organization's expense, or degrade the integrity of the application's security boundary.

The scope covered the public-facing web application and its backend API endpoints. Smart contracts, onchain components, and underlying infrastructure (Cloudflare configuration, RPC provider accounts, CI/CD pipelines) were not in scope, though findings were surfaced where weaknesses in the application create exposure in adjacent systems. No active users or production data were targeted. Testing was conducted against staging and local environments with production-equivalent configuration where feasible.

Testing methodology

Testing combined external (black-box) and white-box approaches.

The external phase was conducted from an unauthenticated, remote attacker position with no prior knowledge of the application beyond what is publicly observable. It focused on endpoint discovery, authentication and authorization boundary testing, input handling, CORS and origin enforcement, rate-limit and geo-location header handling, and abuse of proxied upstream services. This phase established which issues are practically reachable by an unauthenticated attacker on the open internet.

The white-box phase was conducted with full access to the application source code, including frontend (Vue/Nuxt), backend API handlers, and build configuration. It focused on tracing user-controlled input from request ingress through to rendering sinks, reviewing the trust model applied to proxied external data, auditing the RPC proxy's method allow-list and origin-handling logic, and reviewing the client-side OFAC screening composable for reactive-timing issues. Source review was used both to confirm findings surfaced during external testing and to identify issues that would be difficult or uneconomical to discover through black-box probing alone.

Severity rating

Critical – Direct, practical path to significant user-fund loss, full account takeover, or organization-wide compromise. Exploitable remotely with low effort and affects many users.

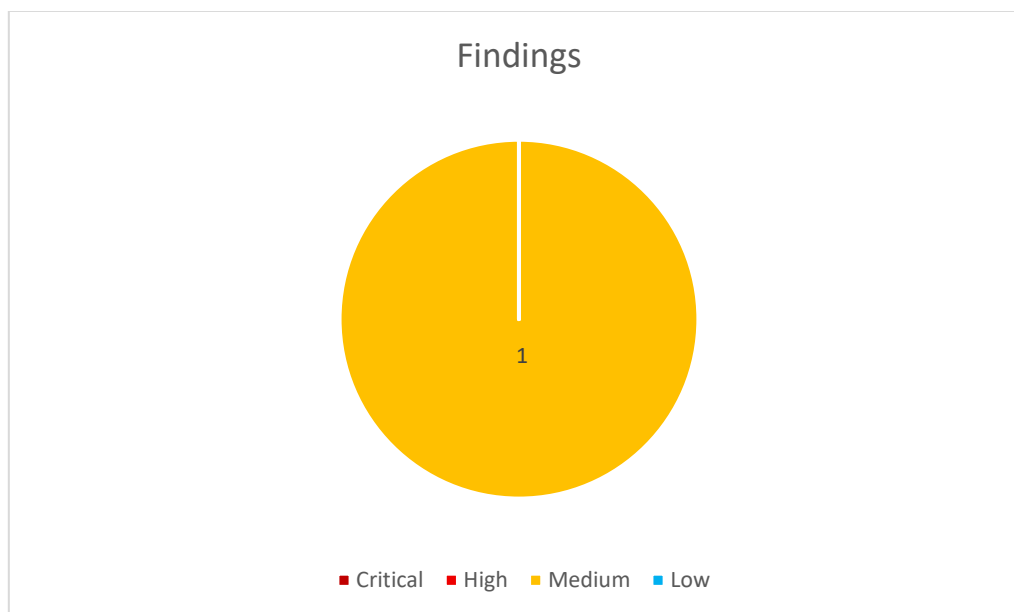
High – Material impact on the organization (regulatory, financial, or reputational), unauthenticated exploitation, or clear bypass of a security control. Does not always require user interaction.

Medium – Meaningful weakening of a control, but either mitigated in the production environment, gated by infrastructure, or limited in impact to indirect cost.

Low – Narrow or edge-case issues that are theoretically exploitable but depend on precise timing, unlikely sequencing, or minor impact relative to compensating controls.

Control effectiveness summary

VectorSec Penetration Testing Report



In aggregate, infrastructure-edge controls and server-side enforcement are doing the security-relevant work, while the frontend carries a thin layer of compliance logic that does not fully hold under adversarial conditions.

Strategic recommendations

The findings in this report address specific weaknesses identified during the engagement. The recommendations in this section are programmatic rather than per-finding. They are intended to complement the remediation work already undertaken.

Penetration testing

- Application penetration testing should be conducted on at least an annual cadence, with an additional targeted engagement following any material architectural change, such as a new chain integration, a change to the authentication model, the introduction of a new backend service, or a significant rewrite of a trust-sensitive component.
- Infrastructure penetration testing should also be conducted annually and should be scoped separately from application testing. The current engagement explicitly excluded Cloudflare configuration, RPC provider accounts, CI/CD pipelines, and origin network exposure.
- Between annual engagements, automated vulnerability scans should be run at least quarterly against both application and infrastructure scopes. Scanning does not replace manual testing, it will not find the classes of issue that require chaining of vulnerabilities or any business logic issues. Scan output should feed into the same remediation tracking process as manual findings.

Secure SDLC

- Running SAST at pre-commit or as a mandatory CI stage on the frontend codebase, with a rule set tuned for Vue/Nuxt rendering sinks and for web-platform URL handling, is the highest-leverage single change that can be made to the development pipeline.
- Secret scanning should be enabled in CI and, where available, at the pre-commit hook.
- Beyond automation, changes to security-sensitive code paths, authentication, ingress header handling, the RPC proxy's method allow-list, rendering of data sourced from external repositories, and any control that gates compliance decisions should require explicit security review as part of the merge process.

Defense in depth

- Any control that gates a compliance or security decision should be implemented as fail-closed by default.
- Content Security Policy should be tightened across the application. A tightly-scoped CSP is a cheap control that substantially reduces the blast radius of any future injection issue.
- A bug bounty or responsible-disclosure program, if one is not already in place, provides continuous external coverage between annual penetration tests and gives researchers who would otherwise have no authorised channel a safe way to report issues. Even a lightweight responsible-disclosure policy, a public security.txt, a monitored security contact, and a clear statement of safe-harbour intent captures most of the value at minimal operational cost and

VectorSec Penetration Testing Report

is a strong signal to the broader security community that reports are welcome. A paid bounty program can be layered on top later once the intake and triage process is established.

Vulnerability details



Medium severity findings

1.1 TRM screening race condition

Finding details

The wallet-connection watcher fires off the TRM screening promise without awaiting it, so the dApp considers the wallet "connected" and renders all wallet-gated UI before the screening verdict returns. If an attacker can stall or drop the outbound `/api/screen-address` request, the screening promise never resolves and the flagged wallet retains a fully-functional session indefinitely.

Composables/useWagmi.ts:66-73

```
watch(wagmiAddress, (address, oldAddress) => {
  if (address && address !== oldAddress) {
    screenConnectedAddress(address)    // ← NOT awaited
  }
  if (!address && oldAddress) {
    resetScreeningCache()
  }
}, { immediate: true })
```

Composables/useAddressScreen.ts:42-52

```
const vpnIsUsed = await detectVpn()
const isRestricted = await screenAddress(address, vpnIsUsed)
if (isRestricted) {
  await disconnect()
  showBlockedModal(address)
}
```

Vue reactivity sets `isConnected.value = true` on the first observation of address. There is no `screeningPassed` gate downstream - every page that conditions on `isConnected` (`lend`, `borrow`, `position`, `swap`) renders normally during the round-trip window.

Security Impact

- Compliance bypass. TRM screening is the dApp's only mechanism enforcing the operator's sanctions/abuse list.
- No server-side enforcement to backstop it. `/api/tenderly/simulate` and `/api/rpc/[chainId]` don't re-check the calling wallet against TRM - they have no notion of which wallet is calling, since the dApp has no SIWE / signed-session auth. Server-side rate limits exist but are wallet-agnostic.

VectorSec Penetration Testing Report

- The disconnect doesn't undo on-chain action. Transactions broadcast are via the user's own wallet RPC, not via the dApp's `/api/rpc/*`. Once the wallet broadcasts, the disconnect is cosmetic.

Remediation

- Gate every wallet-dependent surface on a `screeningPassed` ref, not just `isConnected`.